Appendix of "TempFlash: GPU-Resident, Single-Launch Training of Temporal Graph Neural Networks"

Weiwei Ye
The University of Tokyo
Japan
wwye@g.ecc.u-tokyo.ac.jp

Yankai Chen
MBZUAI
United Arab Emirates
yankaichan3@gmail.com

Zhen Wang
Huzhou Normal University
China
zhenwangrs@gmail.com

Dongyuan Li
The University of Tokyo
Japan
lidy@csis.u-tokyo.ac.jp

This document is the appendix of the paper. Sections 1–8, Tables 1 and 2, Figures 1–8 and Equations (1)–(7) refer to the paper; the works cited here are listed at the end of this appendix.

A PROTOCOL DETAILS

Data. All datasets are preprocessed with the scripts released with [5]: events are sorted by time, timestamps are shifted to start at zero, nodes are renumbered, and the stream is split chronologically into 70% training, 15% validation and 15% test events; GDELT keeps the chronological split of TGL's release, with 67%, 18% and 14% of its events. All frameworks read the same event files and feature tables; ETC and TempFlash also read the same temporal CSR and the same batch boundaries, which adaptive batching produces for every phase. LastFM, Wiki-Talk and Stack-Overflow have no features of their own and use random fp32 features drawn from a standard normal distribution, as in [5], and GDELT uses its binary node and edge features. The T-CSR of LastFM contains both directions of every event (2,586,206 entries), and those of Wiki-Talk, Stack-Overflow and GDELT contain one entry per event (7,833,139, 63,497,049 and 191,290,882 entries), as in the files of TGL and ETC.

Wiki-Talk edge features. The dataset table of [5] lists edge features of dimension 172 for Wiki-Talk, and ETC's training loop indexes the edge feature table unconditionally when it builds mails, but its released data loader creates only node features for this dataset. We therefore draw a $7{,}833{,}139 \times 172$ table of standard normal values, exactly as the loader does for Stack-Overflow, and apply the same one-line change to ETC. All frameworks thus process edge and node feature tables of identical dimensions.

Model selection and testing. The checkpoint with the better validation AP of epochs three and four (counting from zero) is restored and evaluated once on the test events; ties keep the earlier epoch. TGL, ETC, SIMPLE and TempFlash all use this rule. Memory-based models replay the training and validation streams before testing, so that test predictions see the memories of all preceding events, as in the reference. Earlier epochs need not be validated, because the memories, the mailboxes and the sampler pointers are reset at the start of every epoch. Seeds 0, 1 and 2 initialize Python, NumPy and PyTorch, including the GPU generator that draws the negative destinations and the dropout masks.

Average precision. AP is computed per evaluation batch over the positive and negative scores and averaged over batches, as in prior work [5, 20]. TempFlash evaluates the definition of scikit-learn on the GPU: scores are sorted in descending order, and AP is the sum over groups of tied scores of the increase in recall times the precision at the end of the group. On the evaluation batches of all workloads, the result differs from scikit-learn by less than 10^{-8}.

B PROOF OF PROPOSITION 1

Reference semantics. We restate the recent sampler of TGL with one history and one layer, as in our configuration. For node v with T-CSR segment $[o_v, o_{v+1})$, whose timestamps are non-decreasing, the pointer p_v is set to o_v at every reset. For a batch with roots (v_r, t_r), $r = 1, \ldots, R$, and fp32 thresholds $\theta_r = \mathrm{fl}(t_r - 10^{-7})$, the sampler first updates the pointers of all roots and then samples all roots. The update for root r scans $j = p_{v_r}, p_{v_r} + 1, \ldots$ and stops at the first j with $ts_j > \theta_r$; it sets $p_{v_r} \leftarrow j - 1$ if $j \neq p_{v_r}$, leaves the pointer unchanged otherwise, and sets it to $o_{v_r+1} - 1$ if the scan reaches the end of the segment. Updates of the same node are serialized by a lock in an arbitrary order. Sampling root r visits $k = p_{v_r}, p_{v_r} - 1, \ldots$ down to, but excluding, $\max(o_{v_r}, p_{v_r} - K)$ and outputs entry k if $ts_k < \theta_r$, in the order in which the entries are visited, i.e., from the most recent one backward.

Lemma 1 (one update). *Let u_r be the first index of the segment of v_r whose timestamp exceeds θ_r, or o_{v_r+1} if there is none. One update for root r maps p_{v_r} to* $\max(p_{v_r}, u_r - 1)$. The entries with timestamps above θ_r form the suffix $[u_r, o_{v_r+1})$ of the sorted segment. If $u_r > p_{v_r}$, the scan stops at $j = u_r \neq p_{v_r}$, or reaches the end of the segment if $u_r = o_{v_r+1}$, and in both cases the pointer becomes $u_r - 1$. If $u_r \leq p_{v_r}$, the scan stops at its first entry and the pointer is unchanged; for an empty segment, nothing is scanned and $u_r - 1 < o_{v_r} = p_{v_r}$. In every case, the result is $\max(p_{v_r}, u_r - 1)$.

Lemma 2 (order independence). *After the updates of a batch,* $p_v = \max\left(p_v^{\mathrm{old}}, \max_{r:v_r=v}(u_r - 1)\right)$ *for every order of the updates.* By Lemma 1, every update applies a map $x \mapsto \max(x, c_r)$, and such maps commute and compose to $x \mapsto \max(x, \max_r c_r)$, which is exactly the batch-level pointer update of Eq. (4).

Lemma 3 (search). *The upper bound of the query* $(v_r \ll 32) \mid \varphi(\theta_r)$ *in the key array is u_r.* The map φ of Eq. (2) orders all non-NaN fp32 values as the floating-point comparison does, except that it places -0 below $+0$; timestamps shifted to start at zero and thresholds obtained by an fp32 subtraction never take the value -0. Hence, for an entry i of node v_r, κ_i exceeds the query if and only if $ts_i > \theta_r$, while entries of other nodes compare like their node identifiers. Since the key array is sorted, its first entry above the query is u_r.

Proof of Proposition 1. The GPU sampler computes θ_r with the same fp32 subtraction. By Lemmas 2 and 3, the scatter with a maximum reduction yields the reference pointers after the update phase of every batch, by induction over the batches since the last reset. For root r, the reference outputs the entries $p - j$ for $j = 0, \ldots, m - 1$ with $m = \min(p - o_{v_r}, K)$ and $p = p_{v_r}$ whose timestamps are smaller than θ_r, in this order. Slot j of the GPU sampler reads entry $p - j$ and is valid under exactly these two conditions, so the valid slots, read row by row, form the reference sequence. Neighbor identifiers,

Table 3: Commit semantics: how often an execution keeps an update other than the last occurrence (CPU: count; GPU and Write: % of the duplicated nodes).

Data	Model	Seq.	Batches	Updates	Dup.	CPU	GPU	Write
LastFM	TGN	mails	768	1.81M	296.2K	0	55.9%	57.1%
LastFM	TGN	memories	768	1.81M	296.2K	0	57.9%	58.6%
LastFM	APAN	mails	768	11.70M	678.0K	0	80.8%	81.2%
Wiki-Talk	TGN	mails	2,589	10.97M	1.64M	0	66.3%	67.0%
Wiki-Talk	TGN	memories	2,589	10.97M	1.64M	0	57.4%	58.5%
Wiki-Talk	APAN	mails	2,589	65.34M	6.33M	0	77.9%	78.1%
Stack-Ovf.	TGN	mails	300	1.49M	243.1K	0	61.8%	62.3%
Stack-Ovf.	TGN	memories	300	1.49M	243.1K	0	63.8%	64.5%
GDELT	TGN	mails	300	1.58M	148.4K	0	79.7%	80.0%
GDELT	TGN	memories	300	1.58M	148.4K	0	80.1%	80.6%

event identifiers and timestamps are read from the same entries, and time deltas are computed with the same fp32 subtraction. □

C CORRECTNESS TESTS

Sampler. Besides the four graphs of Table 1 (§5.3), we compared the GPU sampler with TGL's C++ sampler on 2,000 random graphs whose events are drawn so that the corner cases of the reference rule occur often: few distinct timestamps, many ties, batches whose roots repeat, nodes without or with only a few earlier events and roots whose threshold equals a stored timestamp. Each graph is processed in two passes with a reset in between, with $K \in \{1, 2, 3, 10\}$ and with one or eight sampler threads. Over 41,404 batches with 1.12M roots and 963.6K sampled neighbors, which contain 397,571 entries whose timestamp equals the threshold, 38,789 batches with repeated roots, 248,918 excluded first entries, 16,752 empty and 431,282 short histories, the two samplers return 0 differing neighbors, event identifiers, timestamps or orders.

Commit semantics. For every training batch, we built the sequences of candidate updates exactly as the reference does (TGN: mails interleaved per event, memories of all sources and then of all destinations; APAN: mails of all roots and then those to the sampled neighbors of every root) and compared three ways of executing ETC's selection with the last occurrence: on the CPU, on the GPU, and as a plain indexed GPU write without selection. Per sequence, Table 3 counts the inspected training batches (on Stack-Overflow and GDELT, 300 strided over the epoch), the updates and the nodes that occur more than once in their batch, summed over batches. It shows that the selection keeps the last occurrence whenever it executes sequentially on the CPU, whereas on the GPU it keeps another update for 56–81% of the duplicated nodes, about as often as a plain indexed write without any selection. Two GPU executions of the same batch also disagree with each other. TempFlash instead commits the last occurrence of every node by construction (§3.5).

Differential test. TempFlash loads the initial parameters of an instance of ETC's model and trains side by side with ETC's training code on the same features, the same adaptive batches and the same negative destinations, with dropout disabled. After every batch, we compare predictions and losses (relative difference), all parameters after the Adam step, the memories and mails of every node touched by the batch (absolute difference), and the memory timestamps, mail timestamps and mailbox ring pointers (exact). The reference executes its selection on the CPU, so that it keeps the last occurrence (Table 3). Over all 768 training batches of a LastFM epoch, the discrete state of TGN and APAN agrees exactly (0 differing timestamps or pointers). Executed as released, i.e., with the selection on the GPU for TGN, the reference differs from TempFlash in 187,586 memory and 163,568 mail timestamps over the same 768 batches, and with the selection on the GPU for APAN in 242,221 and 764,260. With the state of TempFlash re-synchronized to the reference before every batch, the largest relative differences over 300 batches are 1.6×10^{-6} for predictions and 2.4×10^{-6} for parameters for all three models. Two instances of ETC trained from identical parameters agree bitwise (largest prediction difference 0 over 60 batches), and both TempFlash against ETC and ETC against its one-ulp copy exceed a relative prediction difference of 10^{-2} within the first three training batches of the epoch (Figure 5).

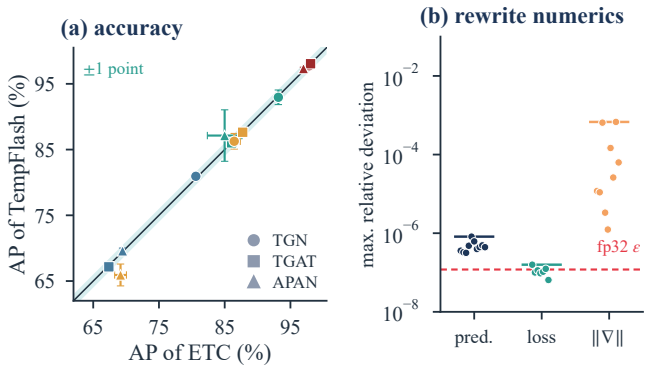


Figure 9: (a) Test AP of TempFlash versus ETC, both with the default configuration (three seeds; bars: one standard deviation; band: ±1 point). (b) Worst relative deviation between the rewritten and the reference operator order.

Rounding against fp64. On 6 synchronized TGN batches, an fp64 copy of the reference model evaluates the same sampled neighborhoods and pre-step states. The median relative prediction errors of the fp32 steps are 3.5×10^{-7} for TempFlash and 4.6×10^{-7} for the reference, and the largest relative gradient errors are 8.8×10^{-7} and 2.2×10^{-5}, i.e., TempFlash rounds no worse than the reference.

Rewrite. For every workload, we start the reference operator order and the project-once-gather-many order from identical parameters and states, train both for 100 to 150 steps and then compare predictions, losses, gradients and committed memories at ten further steps (Figure 9b). The worst relative deviations over all workloads are 8.2×10^{-7} for predictions, 1.6×10^{-7} for losses and 3.4×10^{-3} for gradient norms; the largest gradient deviations stem from ReLU inputs whose sign changes at rounding level. Across an AP range from 65% to 99%, the test AP of TempFlash with the default configuration lies close to that of ETC on the same workload (Figure 9a; mean absolute difference 0.59 points over the twelve workloads with three seeds of both systems).

D HYPERPARAMETERS OF TEMPFLASH

Table 4 lists the nominal batch size, learning rate and dropout with which TempFlash trains every workload of Table 2; all other settings are those of the default configuration (§5.1), including five

Table 4: Hyperparameters of TempFlash in Table 2.

Data	Model	Batch size	Learning rate	Dropout
LastFM	TGN	1,000	$2{\cdot}10^{-4}$	0.2
LastFM	TGAT	5,000	$2{\cdot}10^{-4}$	0.1
LastFM	APAN	600	$1{\cdot}10^{-4}$	0.1
Wiki-Talk	TGN	600	$1{\cdot}10^{-4}$	0.2
Wiki-Talk	TGAT	2,000	$1{\cdot}10^{-4}$	0.1
Wiki-Talk	APAN	1,500	$2{\cdot}10^{-4}$	0.1
Stack-Ovf.	TGN	2,000	$2{\cdot}10^{-4}$	0.2
Stack-Ovf.	TGAT	5,000	$1{\cdot}10^{-4}$	0.1
Stack-Ovf.	APAN	2,000	$1{\cdot}10^{-4}$	0.1
GDELT	TGN	2,500	$2{\cdot}10^{-4}$	0.2
GDELT	TGAT	2,000	$1{\cdot}10^{-4}$	0.1
GDELT	APAN	2,500	$1{\cdot}10^{-4}$	0.1

epochs and seeds 0, 1 and 2, and our code contains one script per workload that runs it with exactly these settings. TempFlash also validates and tests on the adaptive batches of this nominal size; for the models with node memories, which change only between batches, these differ from the batches of the default configuration for APAN on LastFM and TGN on Wiki-Talk. The ablation (Figure 6), the studies of Figures 7 and 8, the complete runs (Figure 4), the accuracy comparison of Figure 9 and the timing sessions (Table 5) use the default configuration, and on the twelve additional graphs, both systems use the configuration of Appendix G.

E TIMING METHODOLOGY

Queues and gates. All timing runs were executed in serialized queues, one per GPU, on identical A100 GPUs. Pairs timed back to back on one GPU are TempFlash and ETC on every workload, and TempFlash and SIMPLE with TGN and TGAT on LastFM and Wiki-Talk and with TGAT on Stack-Overflow. TGL, Orca, Zebra and SIMPLE with APAN on LastFM and Wiki-Talk, with TGN and APAN on Stack-Overflow and with all three models on GDELT were timed in later queues under the same rules, with our accuracy runs paused as described below. Where one of them is the fastest baseline of a workload, TempFlash is at least 1.75× faster, whereas the epoch times of ETC and SIMPLE differ between our sessions by at most a factor of 1.60. A host-wide lock prevents two runs that prepare data with host threads (every baseline, or any process that loads Stack-Overflow or GDELT) from overlapping. A run started only after two consecutive checks, 20 seconds apart, found no active process of another user on its GPU (idle allocations below 4 GB without kernel activity were ignored), a host load of at most 80 in the first session and at most 30 in the later ones, and enough available host memory. A monitor checked the GPU every 15 seconds during the run; runs during which another process became active were discarded and repeated, and so were both runs of a comparison when the check that followed one of them, at most two seconds after its end, found an active process. This second rule matters for short runs, which the monitor otherwise checks only at their start; a scan of the logs of all timing runs found three such cases, all on the additional graphs. In total, 16 runs of TempFlash, 11 of ETC and 0 of SIMPLE were discarded and repeated, and no run of TGL, Orca or Zebra. All timing runs execute with the same elevated CPU scheduling priority (a nice value of −10), which reduces the contention of the host-bound baselines with other users. Our single-threaded runs of TempFlash on the smaller graphs did not take the host lock and could overlap with a timing run on another GPU, and our accuracy runs could share the remaining GPUs at the lowest CPU priority during the first session of every workload and the second of LastFM and Wiki-Talk; later, they were paused, except for single-threaded runs of TempFlash on the smaller graphs.

Measured runs. For LastFM and Wiki-Talk, a session runs, per workload, TempFlash for three epochs, ETC for three (LastFM) or two (Wiki-Talk) epochs, SIMPLE for two epochs, ETC in its all-on-GPU mode, the ablation variants for three epochs each and TempFlash again for three epochs; the other configurations of TempFlash in Table 4, the model-capture variant and a whole-step run with the reference operator order were timed in later queues under the same rules. The third session additionally runs ETC with one and with 32 host threads on LastFM, TempFlash with SIMPLE's fixed batches next to SIMPLE, and on Wiki-Talk the complete engine and the ablation variants on the first 20% of the training batches. For Stack-Overflow and GDELT, a session runs TempFlash for two epochs, ETC for one epoch and TempFlash for one more epoch, and SIMPLE, where it is paired with TempFlash, between two one-epoch runs of TempFlash. The ablation variants of these graphs run on the first 4,000 training batches, i.e., 8–61% of an epoch, and are scaled to a full epoch by their ratio to the complete engine on the same batches, on which the time per batch of the complete engine differs from its full-epoch average by between −11% and +1%. On Wiki-Talk, the same scaling from the first 20% of the batches predicts the measured full-epoch times of twelve variant runs with errors between −2.9% and +9.8%. TGL, Orca and Zebra train three epochs on LastFM and Wiki-Talk; on Stack-Overflow and GDELT, they train the first 1,000 (TGAT: 500) batches of two epochs, whose times are scaled linearly to the full epoch, except for Orca with APAN on GDELT, which trains two complete epochs (Appendix F). In the later queues, SIMPLE trains two epochs. On the twelve additional graphs, TempFlash and ETC each train two epochs, back to back on the same GPU.

Statistics. Every epoch of a timing run is included in its mean; no warm-up epoch is discarded, and the first epoch of ETC takes 0.81–1.38 times as long as its later epochs of the same run. Over the measured epochs of a workload, the coefficient of variation of the epoch time is at most 2.7% for TempFlash and 13.2% for ETC. The two runs of TempFlash that bracket an ETC run differ by at most 0.9% on LastFM and Wiki-Talk; on the larger graphs, their difference reaches 0.7% for TGN on Stack-Overflow and is at most 0.6% otherwise. Where TempFlash uses the default configuration in Table 2, its time is the mean over the epochs of both runs, including the slower one; its other configurations were timed in separate runs of three (LastFM, Wiki-Talk) or two epochs. Between runs on different days, ETC varies more: in the end-to-end runs of Figure 4, its mean epoch time is 6–33% longer than its time in Table 2, whereas that of TempFlash deviates by at most 4% from its session time with the same configuration. TempFlash is timed over its epoch loop, from loading the batch boundaries to the last replay, with device synchronization at both ends; outside it, it only resets its sampler pointers and node states in device memory before every

epoch, and its one-time setup and capture are reported in Table 17. The baselines are timed with the wall-clock time of their training loops, which for ETC includes its sampling pre-pass and its producer thread; its own timer, which omits both, reports 0.6–2.1% less. A background monitor logged GPU utilization and memory, host load and available host memory every 15 seconds.

Timing sessions. ETC's step is bound by the host (§5.5), so its epoch time depends on the activity of other users on the shared server: during the ETC runs of our first session, the mean host load was 32–55 on LastFM and Wiki-Talk and 20–34 on Stack-Overflow and GDELT. We therefore timed LastFM and Wiki-Talk in a second and a third session and Stack-Overflow and GDELT in a second session, with the same code and rules and a load gate of 30 at the start of every run (Table 5). For each workload, Table 2 and the figures report the fastest ETC run over all sessions and thread counts and SIMPLE from its own fastest run; TempFlash with the default configuration and the ablation variants are taken from the session of that ETC run. In the one-epoch runs of Table 6, ETC's TGN epochs on LastFM and Wiki-Talk were up to 11% shorter than in Table 2; SIMPLE is faster there, so no speedup changes. On all twelve workloads, the epoch time of ETC differs between sessions by 7–60% (SIMPLE: 0–30% on five workloads), relative to the faster session, whereas that of TempFlash differs by at most 3.3%, except on GDELT with TGN (4.6%), whose first session coincided with a shortage of host memory. ETC is fastest in the session with the lowest mean host load during its run on all twelve workloads.

Host threads. ETC prepares every batch with CPU threads: its sampler uses the eight threads of its configuration, and its tensor operations use the intra-op thread pool of PyTorch, which by default spans all 80 virtual CPUs and on our shared host makes ETC's epochs up to 1.5× longer. We therefore timed ETC and SIMPLE with TGN on LastFM and Wiki-Talk with 1, 2, 4, 8, 16 and 32 intra-op threads, in two rounds of opposite order with one epoch each and report the mean (Table 6). The rule fixed before these runs keeps the four threads of our first session unless another count has a geometric-mean epoch time over both graphs at least 3% lower, and uses a different count for one graph only if it is at least 3% faster there. For ETC, it yields four threads, and 32 on LastFM. For SIMPLE, it yields four threads, and 32 on LastFM, where we re-timed SIMPLE with this count, with its default and its full budget, between two runs of TempFlash (rows T^{32} and B^{32} of Table 5). On LastFM, the third session also times ETC with one and with 32 threads, and Table 2 takes the fastest of these ETC runs (Table 5). All other ETC and SIMPLE runs, including the batch-size sweep and the utilization traces, use four threads. TempFlash needs no such setting: it runs at full speed on a single host core (§5.5).

Model capture. The model-capture variant of §5.4 executes the data path of TempFlash kernel by kernel with the reference operator order, copies the assembled slots into static input buffers and replays one captured graph per bucket that contains the forward pass, the backward pass and the optimizer update; the state commit runs eagerly after the replay. Its test AP on LastFM equals that of the complete engine within the variation over seeds.

Stage profile. Figure 6(b) is measured with the PyTorch profiler on 100 training batches after 30 warm-up batches with eager launches, which execute the same kernels as the captured step. Every kernel is attributed to the innermost stage range of the host operation that launched it, and kernels launched by the autograd engine form the backward pass; the stages partition the kernel time.

Utilization traces. The traces of Figure 7(a, b) sample nvidia-smi every 50 ms in separate single-epoch runs of ETC and TempFlash and are smoothed by a moving average over 0.5 s; their time axis starts with the epoch, and each curve ends with it.

Table 5: Epoch times (s) and host load in all timing sessions; TempFlash with the default configuration. $S3^T$: ETC with T host threads; B: SIMPLE with a budget for all of its data; T^k (B^k): SIMPLE with k host threads (and the full budget). Bold: runs of Table 2 (TempFlash: the runs around that ETC run). Spd.: speedup over ETC. –: not measured.

Data	Model	Session	Load	ETC	SIMPLE	TempFlash	Spd.
LastFM	TGN	S1	32	20.9	17.8	4.57	4.6×
		S2	20	17.6	16.1	4.57	3.9×
		S3	14	16.8	16.5	4.58	3.7×
		$S3^1$	17	16.3	–	4.58	3.6×
		$S3^{32}$	17	**16.2**	–	**4.58**	3.5×
		B	–	–	13.5	4.56	–
		T^{32}	–	–	14.9	4.57	–
		B^{32}	–	–	**12.6**	4.57	–
	TGAT	S1	35	4.06	5.15	1.65	2.5×
		S2	30	3.36	4.40	1.64	2.0×
		S3	19	3.16	4.05	1.67	1.9×
		$S3^1$	18	**3.15**	–	**1.67**	1.9×
		$S3^{32}$	19	3.32	–	1.67	2.0×
		B	–	–	2.70	1.66	–
		T^{32}	–	–	3.79	1.66	–
		B^{32}	–	–	**2.63**	1.66	–
	APAN	S1	55	18.1	–	2.66	6.8×
		S2	20	11.5	–	2.65	4.4×
		S3	16	11.3	–	2.65	4.3×
		$S3^1$	16	**11.2**	–	**2.65**	4.2×
		$S3^{32}$	16	11.6	–	2.65	4.4×
Wiki-Talk	TGN	S1	44	83.2	130.4	31.8	2.6×
		S2	27	80.4	118.1	31.7	2.5×
		S3	11	**74.2**	100.6	**32.2**	2.3×
		B	–	–	**56.6**	32.1	–
	TGAT	S1	55	28.2	31.3	12.1	2.3×
		S2	38	24.0	33.4	12.1	2.0×
		S3	7	**18.7**	26.0	**12.2**	1.5×
		B	–	–	**13.9**	12.3	–
	APAN	S1	39	113.8	–	15.7	7.3×
		S2	18	123.7	–	15.7	7.9×
		S3	7	**88.9**	–	**15.8**	5.6×
Stack-Ovf.	TGN	S1	34	752.9	–	284.4	2.6×
		S2	23	**650.3**	–	**283.5**	2.3×
	TGAT	S1	31	314.6	385.2	130.2	2.4×
		S2	14	**249.7**	**384.3**	**126.0**	2.0×
	APAN	S1	34	1154.5	–	162.8	7.1×
		S2	30	**1027.1**	–	**159.2**	6.5×
GDELT	TGN	S1	26	1554.4	–	643.2	2.4×
		S2	15	**1453.8**	–	**615.1**	2.4×
	TGAT	S1	20	699.8	–	272.8	2.6×
		S2	13	**596.6**	–	**273.8**	2.2×
	APAN	S1	20	1760.3	–	350.8	5.0×
		S2	14	**1638.3**	–	**347.3**	4.7×

Table 6: Epoch time (s) of ETC and SIMPLE with TGN and T intra-op threads on the host. Bold: fastest.

System	Data	T=1	2	4	8	16	32
ETC	LastFM	14.8	15.8	15.2	15.5	15.2	**14.4**
	Wiki-Talk	77.5	71.3	**69.7**	90.2	73.5	71.0
SIMPLE	LastFM	17.6	15.9	15.5	15.0	15.2	**14.8**
	Wiki-Talk	100.1	100.6	**97.2**	100.2	98.5	99.2

Batch-size sweep. For Figure 8, ETC and TempFlash train two epochs per batch size with all other hyperparameters of the default configuration; the points at the default batch sizes are the session measurements of Table 5. The sweep reports time only.

F BASELINES

Common setup. All baselines run from their released code on our server, read the same event files, splits and feature tables (Appendix A) and are timed under the rules of Appendix E; the accuracy runs of TGL, ETC and SIMPLE train five epochs for each of the seeds 0, 1 and 2. Our changes load these data, record times and results, make the code run on our software stack, adjust model selection and evaluation, or complete missing models, as listed below; every change is marked in the code. Unless noted, accuracy runs on Stack-Overflow and GDELT read one shared read-only copy of the feature tables with the same values as the original tables, instead of one private copy per run.

TGL. We run TGL's code (commit 241b336) with the default configuration, eight sampler threads, four intra-op threads of PyTorch (on LastFM, the faster of 4 and 32) and all data in host memory. The changes load the shared feature tables, index host tables with host indices (current versions of PyTorch no longer accept device indices for host tensors), follow DGL's renamed message functions and add a seed, an epoch timer, a timing-only exit and the model selection of Appendix A. TGL's loader converts boolean feature tables to fp32, which for GDELT's edge features takes 139 GB of host memory; on GDELT, TGL therefore keeps the shared boolean table in all runs and converts the gathered rows of every batch, which yields exactly the same input values.

Orca and Zebra. Both extend the code base of TGN [13] and implement TGN only. We time their released training loops (Orca at commit 2f6d5b4, Zebra at 8e72d65) with their default models, i.e., two graph attention layers over ten neighbors, 100-dimensional memories and embeddings, a learning rate of 10^{-4} and a dropout of 0.4 (Orca) or 0.3 (Zebra), with the nominal batch sizes. Orca reuses cached embeddings without a budget; Zebra samples the top-10 neighbors of its streaming temporal personalized PageRank with the parameters of its documentation ($\alpha = 0.1, 0.1$; $\beta = 0.5, 0.95$). We replaced their data loaders by readers of the same files (both reserve node 0 for padding, so node identifiers are shifted by one) and keep edge-feature tables of more than 8 GB in host memory, gathered per batch (the released code moves the table to the GPU, which does not fit for Stack-Overflow and GDELT). TGAT is the memoryless special case of their architecture: our port removes the node memory and uses the node features as layer-0 representations; it keeps their two attention layers (the default TGAT has one), Orca's embedding reuse and Zebra's PageRank-based neighbors. APAN is ported with the default APAN configuration: ten mails per node, the attention-based memory updater of TGL with layer normalization as the embedding, and delivery of every mail to the node and to its neighbors as sampled by the system (Orca: the ten most recent; Zebra: its top-k neighbors). Mailboxes of more than 8 GB stay in host memory, as in ETC. Their AP in Table 2 is that given in [5] for TGN, the only model they implement; we do not rank it, as they train models different from those of the other frameworks. Table 2 lists them with TGN only, and Table 7 gives the epoch times of our ports, 3.6–29.7 times those of TempFlash.

Table 7: Epoch times (s) of our ports of Orca and Zebra to TGAT and APAN, and of TempFlash (Table 2).

	LastFM		Wiki-Talk		Stack-Overflow		GDELT	
	TGAT	APAN	TGAT	APAN	TGAT	APAN	TGAT	APAN
Orca	9.91	14.5	72.1	228.2	1390.7	2006.8	5095.3	1244.6
Zebra	36.6	41.6	212.4	418.5	2942.1	3208.6	9837.5	4900.3
TempFlash	1.64	3.49	13.2	15.7	126.0	159.2	330.8	347.3

ETC. We run ETC's code (commit db04ac6) with its configurations, its adaptive batching and its model selection; we added the Wiki-Talk edge features (Appendix A), a seed, an epoch timer and a timing-only exit, and chose its host threads by the rule of Appendix E.

Timing on Stack-Overflow and GDELT. On these graphs, one epoch of TGL, Orca or Zebra takes up to hours, and every timing run requires an exclusive GPU and pauses all our accuracy runs. We therefore time these three systems on the first 1,000 (TGAT: 500) training batches of two epochs and scale each epoch linearly to the full number of training batches; for TGL, whose timer records the setup of an epoch before its first batch, that part is kept unscaled. Orca with APAN on GDELT is timed over two complete epochs instead. On Wiki-Talk, where we also timed full epochs, the same scaling from the first 250 (TGN) or 100 (TGAT) batches deviates from the full-epoch time of the three systems by −26% to +30%. In our accuracy runs, which share their GPU and the host with other runs, the fastest complete epoch of these workloads took 0.87–1.55 times the scaled time (sixteen workloads). No scaled time is the fastest baseline of its workload, so the speedup of TempFlash over the fastest baseline does not depend on this linear scaling.

SIMPLE. SIMPLE's released code targets older versions of PyTorch and DGL. A compatibility layer restores two behaviors of these versions on which it relies: reading a host tensor with device indices, which the old versions accepted by copying the indices to the host, and two message functions that DGL has since renamed. To run these workloads, we completed the release as follows. It loads the shared feature tables; its buffer planning, which handles single-layer models only with a one-mail mailbox, is extended to single-layer models without node memory, i.e., to TGAT in the default configuration (the released TGAT configuration has two layers), which then skip writing back buffered node states before evaluation; it saves the placement plan that its preprocessing computes; four errors are fixed, two in its training loop (a tensor used as a truth value and a wrong number of unpacked values in the single-layer edge branch) and two in the evaluation of memory-based models (a misspelled argument and a shape mismatch when buffered mailboxes are written back); and with a budget that holds all edges, which leaves no edge buffer, the edge features are moved to the GPU, as the release already does for node features, where its loader indexes them. The release provides configurations for TGN, TGAT and Orca but not for APAN, although its model, mailbox and planner contain APAN's components; its training loop builds input blocks only for models with an attention layer. We added the missing steps as TGL, from which SIMPLE derives, performs them: the input blocks of APAN's identity layer, the reverse blocks that

deliver mails to the sampled neighbors during evaluation, and per-node ring pointers for buffered mailboxes of more than one mail (the release indexes them with buffer-local identifiers). Its offline planner appends every reuse interval of every data item to Python lists, which needs several hundred GB of host memory for TGN on Stack-Overflow and on GDELT; we compute the same intervals in the same order into preallocated arrays, which leaves every saved plan byte-identical on the workloads where both versions fit into memory, and we keep GPU buffers of GDELT's binary features in fp32. As for ETC, we added a seed option, an epoch timer and a timing-only exit. Apart from the steps added for APAN, none of these changes alters SIMPLE's sampling, batches, data placement or the model computation of its training step.

Configuration and measurement. SIMPLE uses the default configuration of every workload, the two training options of its released configurations (moving the planned buffer intervals to the GPU ahead of time and preloading them) and its default placement budget of 10% of the data volume. For TGN on Stack-Overflow, preloading the plans of all batches would need 255 GB of host memory; there, SIMPLE plans each batch when it processes it, as the other mode of its release does, which yields the same plans and includes their computation in its epoch time. It does not implement adaptive batching and groups the events into fixed batches of the nominal size, so that its epochs have 5–41% more batches than adaptive batching yields (Table 8). Its preprocessing samples the batches of an epoch once and plans the buffer content offline; this takes 1.1–16.0 times as long as one of its training epochs and is not included in Table 2, which favors SIMPLE. The plan depends only on the data, the model, the batch size and the budget, so all seeds of a workload share it. It only decides which rows stay in GPU buffers: SIMPLE samples negatives and neighbors anew for every batch, like TGL, and reads other rows from host memory; on Stack-Overflow, SIMPLE needs up to 156 GB of host memory. SIMPLE selects and tests checkpoints by the rule of Appendix A. Timing runs train two epochs (one for TGAT on Stack-Overflow) without validation, under the timing rules of Appendix E. With a placement budget that holds all of its data and a new offline plan (rows B of Table 5), SIMPLE is faster than with its default budget on all four workloads on which we ran it, with 15–47% shorter epochs; Table 2 uses the faster budget; its accuracy runs use the default one. SIMPLE comes closest for TGAT on Wiki-Talk (1.1×), where most attention slots of TempFlash hold no neighbor but still cost arithmetic and memory traffic in every training step (Appendix H).

Batches and accuracy. To separate SIMPLE's data placement from its batching, we also trained TempFlash with the default configuration and SIMPLE's fixed batches, timed next to SIMPLE (Table 8). With the same batches, TempFlash is 2.2–3.7× faster than SIMPLE with its default budget in the same session on the five workloads on which both ran. On Wiki-Talk, SIMPLE's test AP is 3.9–9.0 points below ETC's; on the other nine workloads on which both have three seeds, it lies within 1.4 points of ETC's, except for APAN on Stack-Overflow, where it is 3.65 points higher. With SIMPLE's fixed batches, the test AP of TempFlash stays within the variation over seeds of its AP with adaptive batches on all five workloads with three fixed-batch seeds. SIMPLE reads the event files and the temporal CSR of TGL and ETC, samples temporal neighbors with TGL's sampler and uses the default configuration and the same model selection as ETC and TempFlash.

Table 8: TempFlash with adaptive batches (adapt.) and SIMPLE's fixed batches (fixed), and SIMPLE: batches per epoch, epoch time (s) and test AP (%, mean over three seeds).

		Batches		Time (s)			AP (%)		
Data	Model	adapt.	fixed	adapt.	fixed	SIMPLE	adapt.	fixed	SIMPLE
LastFM	TGN	768	906	4.58	4.52	16.5	80.93	81.20	81.68
LastFM	TGAT	173	182	1.67	1.67	4.05	67.16	67.44	67.60
Wiki-Talk	TGN	2589	3656	32.2	32.1	100.6	92.95	92.94	84.14
Wiki-Talk	TGAT	847	1097	12.2	11.9	26.0	86.03	86.20	82.15
Stack-Ovf.	TGAT	6528	8890	126.0	125.7	384.3	87.64	87.37	87.70

Systems not compared. SWIFT [6] targets graphs that exceed host memory and stages them from SSDs; its release reuses 90% of the negative samples across batches, which changes the training objective, and requires a modified DGL build, so we did not port it. MSPipe [14] and PipeTGL [11] train with stale memories, Cascade [3] changes the batches by dependency-aware batching, and PRISM [8] and ATLAS [9] change how memories are synchronized with training; these systems therefore train under other protocols than ours. We rank AP only for the frameworks that follow our protocol, which excludes Orca and Zebra. TASER [4] samples on the GPU but replaces the most-recent sampler by an adaptive one, which changes the model inputs, and GNNFlow [19] targets continuous learning on evolving graphs. FAST [1] deduplicates and caches the transferred features, accelerates the CPU sampler and refines the GPU operators, but it keeps the host-centric step that SIMPLE represents in our comparison; we did not port it to our protocol. TGOpt [17] accelerates inference only, and NeutronStream [2], DynaHB [16] and SWASH [15] target window- or snapshot-based dynamic GNNs, which our protocol excludes.

G ADDITIONAL DYNAMIC GRAPHS

Data. Wikipedia and Reddit come from the data directory of TGL and ETC, which contains the JODIE graphs with their 172-dimensional edge features [10, 20]. Enron and UCI come from the TGB distribution [7], which contains no features; as for LastFM, we draw 128-dimensional node and edge features from a standard normal distribution. MOOC, SocialEvo, Flights, CanParl, USLegis, UNtrade, UNvote and Contacts are the processed versions of Poursafaei et al. [12], as distributed with DyGLib [18], with their original edge features and without node features. All graphs are converted with the preprocessing of Appendix A: events are sorted stably by time, timestamps are shifted to start at zero, the stream is split chronologically into 70%, 15% and 15% of the events, and the temporal CSR contains both directions of every event. ETC and TempFlash read exactly the same converted event and feature files.

Protocol. Both systems use the default model configuration (§5.1) with a nominal batch size of 600 events for all three models, a learning rate of 10^{-4}, five epochs, adaptive batching, the model selection of Appendix A and seeds 0, 1 and 2. Table 10 lists all cells with the standard deviation over seeds and the epoch times of both systems, measured back to back on a dedicated GPU (Appendix E).

Table 9: Standard deviation (points) over seeds of the test AP in Table 2.

	LastFM			Wiki-Talk			Stack-Overflow			GDELT		
Framework	TGN	TGAT	APAN	TGN	TGAT	APAN	TGN	TGAT	APAN	TGN	TGAT	APAN
TGL	1.88	0.63	0.47	0.12	0.09	3.22	1.69	0.06	2.01	1.04	0.01	0.37
ETC	0.16	0.56	0.15	0.39	0.06	2.63	0.92	0.17	0.90	0.24	0.00	0.08
SIMPLE	0.26	0.67	3.31	0.10	0.13	0.53	0.50	0.27	0.07	2.82	0.01	0.42
TempFlash	0.86	0.43	0.11	1.36	0.05	0.94	0.40	0.16	1.66	0.55	0.00	0.20

Table 10: Additional graphs: seeds, test AP (%, mean ± standard deviation), epoch time (s) and speedup.

Data	Model	n	ETC AP	TempFlash AP	ETC	TempFlash	Spd.
Wikipedia	TGN	3	97.53 ± 0.27	97.30 ± 0.19	3.14	0.84	3.8×
	TGAT	3	88.64 ± 0.21	88.62 ± 0.25	2.46	0.37	6.6×
	APAN	3	91.07 ± 0.65	91.01 ± 0.27	2.87	0.50	5.7×
Reddit	TGN	3	99.14 ± 0.63	99.50 ± 0.05	18.1	3.72	4.9×
	TGAT	3	96.36 ± 0.25	96.09 ± 0.12	7.86	1.45	5.4×
	APAN	3	98.52 ± 0.60	98.75 ± 0.10	12.1	1.89	6.4×
MOOC	TGN	3	99.34 ± 0.04	99.37 ± 0.04	6.78	1.69	4.0×
	TGAT	3	97.13 ± 0.08	97.14 ± 0.02	5.17	0.72	7.1×
	APAN	3	98.87 ± 0.06	98.89 ± 0.09	5.39	0.98	5.5×
Enron	TGN	3	71.88 ± 2.02	70.84 ± 1.97	2.73	0.55	4.9×
	TGAT	3	70.75 ± 0.67	70.65 ± 0.40	1.95	0.31	6.3×
	APAN	3	52.99 ± 1.54	53.58 ± 1.96	1.85	0.33	5.6×
SocialEvo	TGN	3	80.56 ± 0.05	82.46 ± 0.31	38.1	7.69	5.0×
	TGAT	3	77.94 ± 0.07	77.90 ± 0.02	23.4	3.52	6.7×
	APAN	3	63.96 ± 4.86	60.77 ± 9.76	23.9	4.32	5.5×
UCI	TGN	3	80.15 ± 3.91	76.86 ± 7.22	1.29	0.33	3.9×
	TGAT	3	63.11 ± 1.25	63.17 ± 0.17	0.97	0.20	4.8×
	APAN	3	62.61 ± 2.17	62.47 ± 1.13	1.08	0.21	5.1×
Flights	TGN	3	90.38 ± 1.63	90.85 ± 1.66	32.5	8.43	3.9×
	TGAT	3	82.12 ± 0.08	82.08 ± 0.06	19.6	3.33	5.9×
	APAN	3	87.35 ± 0.36	88.01 ± 1.46	24.0	4.43	5.4×
CanParl	TGN	3	72.09 ± 3.34	72.85 ± 1.44	3.06	0.37	8.3×
	TGAT	3	71.18 ± 0.22	71.26 ± 0.24	0.93	0.19	5.0×
	APAN	3	63.41 ± 0.67	64.76 ± 2.09	0.98	0.22	4.5×
USLegis	TGN	3	74.58 ± 0.68	75.88 ± 0.29	1.29	0.25	5.1×
	TGAT	3	68.08 ± 0.33	68.05 ± 0.59	0.85	0.13	6.4×
	APAN	3	65.66 ± 4.88	64.16 ± 3.98	0.81	0.16	5.1×
UNtrade	TGN	3	65.43 ± 0.59	65.58 ± 0.86	7.46	1.89	4.0×
	TGAT	3	53.37 ± 0.05	53.38 ± 0.04	5.63	0.87	6.5×
	APAN	3	60.01 ± 1.62	60.90 ± 3.32	5.90	1.09	5.4×
UNvote	TGN	3	58.56 ± 1.41	57.96 ± 0.35	14.1	3.84	3.7×
	TGAT	3	50.93 ± 0.01	50.95 ± 0.03	11.0	1.71	6.4×
	APAN	3	55.60 ± 1.27	55.28 ± 1.10	10.9	2.18	5.0×
Contacts	TGN	3	89.97 ± 0.17	89.56 ± 1.12	42.1	10.4	4.1×
	TGAT	3	88.45 ± 0.08	88.38 ± 0.10	28.0	4.59	6.1×
	APAN	3	87.05 ± 1.43	82.48 ± 5.29	24.7	5.83	4.2×

Table 11: Memory-based models on the additional graphs: updates per updated node and batch, deviation of ETC's GPU selection (% of the duplicated nodes) and test AP (%, three seeds) of ETC, ETC-det and TempFlash.

Data	Model	Upd./node	GPU dev.	ETC	ETC-det	TempFlash
Wikipedia	TGN	3.8	55%	97.5	97.5	97.3
	APAN	10.5	82%	91.1	91.2	91.0
Reddit	TGN	2.0	42%	99.1	99.5	99.5
	APAN	5.9	74%	98.5	98.8	98.7
MOOC	TGN	7.9	78%	99.3	99.4	99.4
	APAN	32.8	83%	98.9	98.9	98.9
Enron	TGN	20.1	94%	71.9	71.0	70.8
	APAN	38.0	96%	53.0	53.0	53.6
SocialEvo	TGN	32.4	91%	80.6	82.6	82.5
	APAN	64.7	92%	64.0	62.4	60.8
UCI	TGN	5.4	57%	80.1	75.4	76.9
	APAN	16.5	84%	62.6	63.0	62.5
Flights	TGN	2.9	47%	90.4	88.8	90.9
	APAN	4.4	72%	87.3	88.3	88.0
CanParl	TGN	4.5	52%	72.1	72.6	72.9
	APAN	4.8	52%	63.4	64.3	64.8
USLegis	TGN	15.8	90%	74.6	75.4	75.9
	APAN	47.3	90%	65.7	64.5	64.2
UNtrade	TGN	7.2	74%	65.4	64.9	65.6
	APAN	7.2	74%	60.0	59.4	60.9
UNvote	TGN	12.5	79%	58.6	56.8	58.0
	APAN	12.5	84%	55.6	55.5	55.3
Contacts	TGN	6.1	80%	90.0	90.2	89.6
	APAN	16.4	87%	87.0	82.5	82.5

Accuracy. Across the 36 workloads, the test AP of the two systems differs by 0.69 points on average and by at most 4.57 points; the difference is at most twice the larger of their standard deviations over seeds, or one point, on 35 of them. The two differ by more than this on SocialEvo with TGN (+1.9 points).

Does ETC's GPU selection matter? ETC's released code executes the selection that commits node states on the GPU, where it keeps another update than the last for 42–96% of the duplicated nodes on these graphs (Table 11); on graphs with fewer than 1,000 nodes, each batch updates the same nodes many times, so the commit rule decides which update survives. We therefore ran a pre-registered variant of ETC's code, ETC-det, in which the six selections compute the last position of every node with an order-independent reduction, i.e., the rule that TempFlash commits; nothing else changes. On all 24 workloads, ETC-det differs from the released code of ETC by 1.01 AP points on average and by at most 4.79; by the pre-registered criterion of the previous paragraph, the two agree on 23 of them and differ on SocialEvo with TGN. By the same rule, ETC-det agrees with TempFlash on 23 of them, including SocialEvo with TGN, where TempFlash and the released code disagree, and differs on UNvote with TGN (1.1 points); on average, it lies 0.55 points from TempFlash, against 1.00 points for the released code, whose selection runs on the GPU. We also ran ETC-det on Stack-Overflow with APAN. There, TempFlash reaches 65.92% AP (standard deviation over seeds 1.7) and ETC's released code 69.15% (0.9), which agrees with TempFlash by this rule; ETC-det reaches 67.93% (0.8), 2.01 points above TempFlash, and also agrees with TempFlash by the same rule.

H IMPLEMENTATION DETAILS

Sampling time. Table 13 times the sampling of the training batches of one epoch with TGL's C++ sampler on 1, 8 and 32 host threads, and with the GPU sampler of TempFlash, launched kernel by kernel (eager) or replayed from a captured graph. Both samplers

Algorithm 1 Captured training step for bucket B

Require: cursor c; batch starts $lo[\cdot]$ and sizes $sz[\cdot]$; event arrays; T-CSR; tiers; states

$n \leftarrow sz[c]$; $ok \leftarrow [0, B) < n$; $ev \leftarrow \min(lo[c] + [0, B), |E| - 1)$ ▷ device cursor

$(\mathbf{src}, \mathbf{dst}, \mathbf{t}) \leftarrow$ events$[ev]$; $\mathbf{neg} \leftarrow$ randint$(|V|, B)$

roots $\leftarrow [\mathbf{src}; \mathbf{dst}; \mathbf{neg}]$ at times $[\mathbf{t}; \mathbf{t}; \mathbf{t}]$

$(nbr, eid, \Delta t, mask) \leftarrow$ Sample(roots, ok) ▷ §3.2

gather features, memories and mails in slot layout ▷ §3.4, §3.6

$\mathcal{L} \leftarrow$ MaskedBCE(Model$(\cdot)$, ok); $\nabla\mathcal{L}$; Adam step ▷ §3.3

Commit(new memories and mails, ok) ▷ §3.5

$c \leftarrow c + 1$

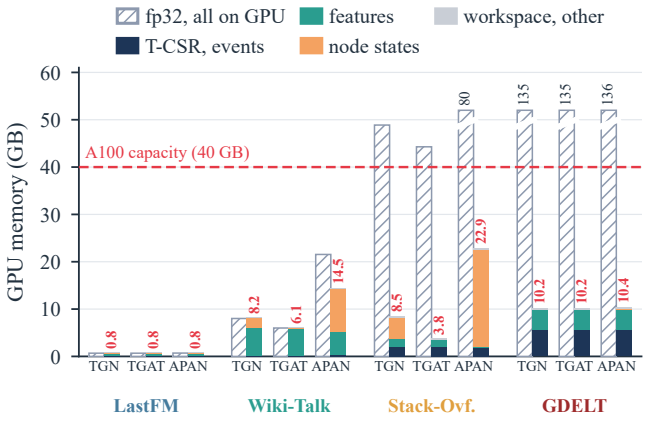


Figure 10: GPU memory per workload with all data in fp32 (hatched) and as allocated by TempFlash (solid).

process the same roots and return the same neighbors, whose total number per epoch is given in column Nbrs.

Redundant reads. Table 15 quantifies the price of not deduplicating over 300 training batches strided over an epoch: for the nominal batch size B, it gives the share of valid slots, the distinct edge rows among them and the factor by which a deduplicating system would read fewer edge rows, and the same for node rows (roots and neighbors). Edge rows recur because the most recent events of a node appear in the neighborhoods of all its roots, and node rows recur most on graphs with few nodes such as LastFM. A deduplicating gather would read 1.8–5.6× fewer edge rows, including 2.1× on Stack-Overflow, whose edge features are read over PCIe. A deduplication with a sort or hash table of fixed capacity per batch could also be captured; we leave it to future work.

Empty slots. The slot layout also computes every slot: the attention of a root covers all K slots, of which only the valid ones hold a neighbor (Table 15), whereas message-flow blocks contain only the sampled neighbors. This costs most for TGAT, whose step consists almost entirely of this attention and in which only 29–47% of the slots are valid. On Wiki-Talk, TempFlash thus computes 3.4× as many key and value rows as ETC, and the forward and backward passes of its step take 13.1 ms, more than the 10.1 ms of ETC's whole computation phase per batch; TGAT is also the model that gains least from removing the host (Table 2). Compacting the valid slots into fixed-capacity buffers would remove this work, but the capacity must then cover the largest number of valid slots, which depends on the random negatives of a batch.

Padding. Over all batches of the three phases (Table 16), with a single bucket, padding to the largest batch adds up to 47.5% rows; four buckets reduce this to at most 11.0%, and eight buckets would save little more (at most 10.6%). Padded rows are masked out and therefore cost only arithmetic, not accuracy.

Memory. Table 12 splits the device memory of TempFlash into the T-CSR (Entries: its number of entries; T-CSR: its size including the sorting keys), the event arrays, node and edge features, node states (memories and mailboxes) and other tensors, including the workspace of a step; Host gives the edge features kept in the zero-copy host tier, i.e., in pinned host memory.

Setup. Loading the event stream, building the T-CSR, the keys and the tiers, computing the batch boundaries and capturing all graphs take at most 157 s per workload (Table 17), of which capture takes 1.0–2.4 s. ETC also preprocesses its data before training, and neither preprocessing is included in the epoch times of Table 2.

REFERENCES

[1] Yushu Cai, Qingrui Zhu, Lei Liu, Kai Sheng, Hao Chen, and Xin He. 2026. FAST: A Holistic Framework for Optimizing Memory-I/O, Computation, and Sampling in Temporal GNN Training. In *Proceedings of the 55th International Conference on Parallel Processing (ICPP)*. https://doi.org/10.1145/3832810.3832847

[2] Chaoyi Chen, Dechao Gao, Yanfeng Zhang, Qiange Wang, Zhenbo Fu, Xuecang Zhang, Junhua Zhu, Yu Gu, and Ge Yu. 2023. NeutronStream: A Dynamic GNN Training Framework with Sliding Window for Graph Streams. *Proceedings of the VLDB Endowment* 17, 3 (2023), 455–468. https://doi.org/10.14778/3632093.3632108

[3] Yue Dai, Xulong Tang, and Youtao Zhang. 2025. Cascade: A Dependency-aware Efficient Training Framework for Temporal Graph Neural Network. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. https://doi.org/10.1145/3676641.3716250

[4] Gangda Deng, Hongkuan Zhou, Hanqing Zeng, Yinglong Xia, Christopher Leung, Jianbo Li, Rajgopal Kannan, and Viktor Prasanna. 2024. TASER: Temporal Adaptive Sampling for Fast and Accurate Dynamic Graph Representation Learning. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. https://doi.org/10.1109/IPDPS57955.2024.00087

[5] Shihong Gao, Yiming Li, Yanyan Shen, Yingxia Shao, and Lei Chen. 2024. ETC: Efficient Training of Temporal Graph Neural Networks over Large-Scale Dynamic Graphs. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1060–1072. https://doi.org/10.14778/3641204.3641215

[6] Rui Guo, Zezhong Ding, Xike Xie, and Jianliang Xu. 2025. SWIFT: Enabling Large-Scale Temporal Graph Learning on a Single Machine. *Proceedings of the ACM on Management of Data* 3, 4 (2025). https://doi.org/10.1145/3749184

[7] Shenyang Huang, Farimah Poursafaei, Jacob Danovitch, Matthias Fey, Weihua Hu, Emanuele Rossi, Jure Leskovec, Michael Bronstein, Guillaume Rabusseau, and Reihaneh Rabbany. 2023. Temporal Graph Benchmark for Machine Learning on Temporal Graphs. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. https://doi.org/10.52202/075280-0099

[8] Md Ashraful Islam, Hojae Son, Suhaas Kiran Doddagaddavalli Gangadharaiah, and Marco Serafini. 2026. PRISM: A Training System to Unlock the Potential of Temporal Graph Learning Through Staleness Avoidance. *Proceedings of the VLDB Endowment* (2026). https://doi.org/10.14778/3819518.3819544

[9] Yongchun Jiang, Yongchao Liu, Kaifan Jia, and Heng Zhang. 2026. ATLAS: Enabling an Asynchronous State-Aware Pipeline for Efficient Training of Temporal Graph Neural Networks. In *Proceedings of the 55th International Conference on Parallel Processing (ICPP)*. https://doi.org/10.1145/3832810.3832844

[10] Srijan Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*. https://doi.org/10.1145/3292500.3330895

[11] Jun Liu, Bingqian Du, Ziyue Luo, Sitian Lu, Qiankun Zhang, and Hai Jin. 2025. PipeTGL: (Near) Zero Bubble Memory-Based Temporal Graph Neural Network Training via Pipeline Optimization. *Proceedings of the VLDB Endowment* (2025). https://doi.org/10.14778/3742728.3742760

[12] Farimah Poursafaei, Shenyang Huang, Kellin Pelrine, and Reihaneh Rabbany. 2022. Towards Better Evaluation for Dynamic Link Prediction. In *Advances in*

Table 12: Device memory (GB) allocated by TempFlash per workload, by array, and the edge features kept in the zero-copy host tier. ^{p}: bit-packed; ^{f}: factorized mailbox.

Data	Model	Entries	T-CSR	Events	Node feat.	Edge feat.	States	Other	Total	Host
LastFM	TGN	2.59M	0.05	0.01	0.00	0.62	0.00	0.12	0.80	0.0
LastFM	TGAT	2.59M	0.05	0.01	0.00	0.62	0.00	0.09	0.77	0.0
LastFM	APAN	2.59M	0.05	0.01	0.00	0.62	0.03	0.10	0.80	0.0
Wiki-Talk	TGN	7.83M	0.16	0.09	0.73	5.02	2.03	0.17	8.20	0.0
Wiki-Talk	TGAT	7.83M	0.16	0.09	0.73	5.02	0.00	0.13	6.13	0.0
Wiki-Talk	APAN	7.83M	0.16	0.09	0.00	5.02	9.07^{f}	0.13	14.47	0.0
Stack-Ovf.	TGN	63.50M	1.22	0.71	1.67	0.00	4.63	0.27	8.50	40.7
Stack-Ovf.	TGAT	63.50M	1.22	0.71	1.67	0.00	0.00	0.21	3.81	40.7
Stack-Ovf.	APAN	63.50M	1.22	0.71	0.00	0.00	20.70^{f}	0.22	22.85	40.7
GDELT	TGN	191.29M	3.56	2.14	0.03	4.10^{p}	0.03	0.39	10.24	0.0
GDELT	TGAT	191.29M	3.56	2.14	0.03	4.10^{p}	0.00	0.37	10.19	0.0
GDELT	APAN	191.29M	3.56	2.14	0.03	4.10^{p}	0.24	0.35	10.42	0.0

Table 13: Time (s) to sample the training batches of one epoch with TGL's C++ sampler and with the GPU sampler of TempFlash. Nbrs.: sampled neighbors per epoch.

		TGL sampler (threads)			TempFlash sampler	
Data	Nbrs.	1	8	32	eager	captured
LastFM	16.2M	1.5	1.2	1.2	0.58	0.12
Wiki-Talk	57.6M	10.3	5.4	5.7	2.08	0.50
Stack-Ovf.	673.6M	99.1	49.7	49.6	11.64	3.25
GDELT	1586.6M	165.3	119.9	198.8	30.41	6.93

Table 14: Kernels and wall time per training step with the default configuration and all data on the GPU, launched one by one (Eager) or replayed from a captured graph (Replay).

Data	Model	Kernels	Eager (ms)	Replay (ms)	Ratio
LastFM	TGN	474	8.0	6.0	1.3×
	TGAT	349	10.5	9.7	1.1×
	APAN	377	6.6	3.4	1.9×
Wiki-Talk	TGN	468	13.0	12.4	1.0×
	TGAT	345	15.0	14.5	1.0×
	APAN	392	6.7	6.1	1.1×
Stack-Ovf.	TGN	470	17.9	17.3	1.0×
	TGAT	344	19.1	18.6	1.0×
	APAN	393	10.0	9.4	1.1×
GDELT	TGN	485	13.4	12.7	1.1×
	TGAT	350	11.5	11.0	1.0×
	APAN	379	7.6	7.0	1.1×

Table 15: Redundancy of the slot layout: valid slots and distinct edge and node rows, with the factor by which a deduplicating gather would read fewer rows.

Data	Models	B	Valid	Edges	Reads	Nodes	Reads
LastFM	TGN/APAN	1,000	58%	40%	2.5×	6%	15.9×
LastFM	TGAT	5,000	40%	18%	5.6×	2%	40.8×
Wiki-Talk	TGN/APAN	1,500	34%	39%	2.6×	21%	4.8×
Wiki-Talk	TGAT	5,000	29%	32%	3.1×	18%	5.5×
Stack-Ovf.	TGN/APAN	2,000	50%	48%	2.1×	27%	3.7×
Stack-Ovf.	TGAT	5,000	47%	44%	2.3×	24%	4.3×
GDELT	TGN/APAN	2,500	41%	57%	1.8×	10%	10.0×
GDELT	TGAT	5,000	37%	53%	1.9×	8%	11.9×

Table 16: Padding (% of rows) of all batches with 1, 2, 3, 4 or 8 batch-size buckets; TempFlash uses four.

Data	Models	Batches	Sizes	1	2	3	4	8
LastFM	TGN/APAN	1,101	355–1,368	16.5	7.0	6.5	6.5	5.5
LastFM	TGAT	251	917–5,328	3.4	2.5	2.5	2.4	2.4
Wiki-Talk	TGN/APAN	3,984	1,370–2,771	40.9	21.5	12.2	11.0	10.6
Wiki-Talk	TGAT	1,293	30–8,935	47.5	25.6	10.7	9.6	9.0
Stack-Ovf.	TGN/APAN	23,439	1,118–3,453	27.5	15.6	11.1	9.4	8.2
Stack-Ovf.	TGAT	9,997	3,506–7,584	19.4	11.5	9.0	8.0	7.3
GDELT	TGN/APAN	72,702	728–2,719	3.3	2.2	2.2	2.2	2.2
GDELT	TGAT	37,073	763–5,292	2.6	1.1	0.8	0.8	0.8

Table 17: One-time cost of TempFlash: setup and capture (s), epoch time (s) and their share in a five-epoch run.

Data	Model	Setup	Capture	Epoch	Share
LastFM	TGN	4.8	1.7	4.58	22.1%
LastFM	TGAT	4.2	1.7	1.67	41.6%
LastFM	APAN	3.9	1.6	2.65	29.6%
Wiki-Talk	TGN	23.3	2.1	32.2	13.6%
Wiki-Talk	TGAT	28.1	2.3	12.2	33.1%
Wiki-Talk	APAN	23.7	2.0	15.8	24.6%
Stack-Ovf.	TGN	128.9	2.2	283.5	8.5%
Stack-Ovf.	TGAT	157.5	2.4	126.0	20.2%
Stack-Ovf.	APAN	149.3	2.0	159.2	16.0%
GDELT	TGN	83.2	1.1	615.1	2.7%
GDELT	TGAT	61.4	1.6	273.8	4.4%
GDELT	APAN	76.9	1.0	347.3	4.3%

Neural Information Processing Systems 35 (NeurIPS), Datasets and Benchmarks Track. https://doi.org/10.52202/068431-2386

[13] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. *arXiv preprint arXiv:2006.10637* (2020).

[14] Guangming Sheng, Junwei Su, Chao Huang, and Chuan Wu. 2024. MSPipe: Efficient Temporal GNN Training via Staleness-Aware Pipeline. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. https://doi.org/10.1145/3637528.3671844

[15] Zhen Song, Yu Gu, Tianyi Li, Yushuai Li, Qing Sun, Yanfeng Zhang, Christian S. Jensen, and Ge Yu. 2025. SWASH: A Flexible Communication Framework with Sliding Window-Based Cache Sharing for Scalable DGNN Training. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26. https://doi.org/10.1145/3725360

[16] Zhen Song, Yu Gu, Qing Sun, Tianyi Li, Yanfeng Zhang, Yushuai Li, Christian S. Jensen, and Ge Yu. 2024. DynaHB: A Communication-Avoiding Asynchronous Distributed Framework with Hybrid Batches for Dynamic GNN Training. *Proceedings of the VLDB Endowment* 17, 11 (2024), 3388–3401. https://doi.org/10.14778/3681954.3682008

[17] Yufeng Wang and Charith Mendis. 2023. TGOpt: Redundancy-Aware Optimizations for Temporal Graph Attention Networks. In *Proceedings of the 28th ACM*

SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP). https://doi.org/10.1145/3572848.3577490

[18] Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. 2023. Towards Better Dynamic Graph Learning: New Architecture and Unified Library. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[19] Yuchen Zhong, Guangming Sheng, Tianzuo Qin, Minjie Wang, Quan Gan, and Chuan Wu. 2023. GNNFlow: A Distributed Framework for Continuous Temporal GNN Learning on Dynamic Graphs. https://doi.org/10.48550/arXiv.2311.17410 arXiv preprint arXiv:2311.17410.

[20] Hongkuan Zhou, Da Zheng, Israt Nisa, Vasileios Ioannidis, Xiang Song, and George Karypis. 2022. TGL: A General Framework for Temporal GNN Training on Billion-Scale Graphs. *Proceedings of the VLDB Endowment* 15, 8 (2022), 1572–1580. https://doi.org/10.14778/3529337.3529342